

CO3-AT2-Q2:

Name: D.Uday Kiran

Reg no: 192425161

Course Code: CSA0652

Course Name: DAA for Complexity Analysis

Government Analytics Platform Using Merge Sort and Quick Sort

CODE:

```
#include<stdio.h>
```

```
void merge(int a[],int l,int m,int h)
```

```
{
```

```
int i=l,j=m+1,k=l,temp[100];
```

```
while(i<=m&& j<=h)
```

```
{
```

```
if(a[i]<=a[j])
```

```
temp[k++]=a[i++];
```

```
else
```

```
temp[k++]=a[j++];
```

```
}
```

```
while(i<=m)
```

```
temp[k++]=a[i++];
```

```
while(j<=h)
```

```
temp[k++]=a[j++];
```

```
for(i=l;i<=h;i++)
```

```
a[i]=temp[i];
```

```

}

void mergeSort(int a[],int l,int h)

{
if(l<h)
{
int m=(l+h)/2;

mergeSort(a,l,m);

mergeSort(a,m+1,h);

merge(a,l,m,h);
}
}

void quickSort(int a[],int l,int h)

{
if(l<h)
{
int pivot=a[h],i=l-1,j,temp;

for(j=l;j<h;j++)
{
if(a[j]<pivot)
{
i++;

temp=a[i];

a[i]=a[j];

```

```

a[j]=temp;

}

}

temp=a[i+1];
a[i+1]=a[h];
a[h]=temp;

quickSort(a,l,i);
quickSort(a,i+2,h);

}

}

void display(int a[],int n)

{

for(int i=0;i<n;i++)

printf("%d ",a[i]);

}

int main()

{

int a[100],b[100],n;

printf("Enter number of data records: ");

scanf("%d",&n);

printf("Enter dataset values:\n");

for(int i=0;i<n;i++)

{

```

```

scanf("%d",&a[i]);

b[i]=a[i];

}

mergeSort(a,0,n-1);

printf("Merge Sort:\n");

display(a,n);

quickSort(b,0,n-1);

printf("\nQuick Sort:\n");

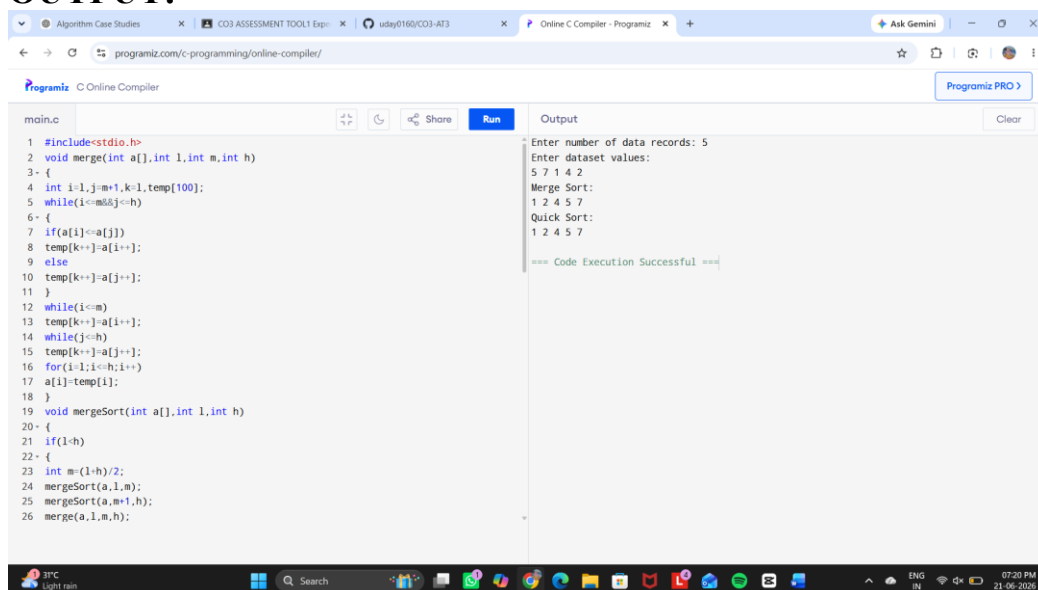
display(b,n);

return 0;

}

```

OUTPUT:



```

main.c
1 #include<stdio.h>
2 void merge(int a[],int l,int m,int h)
3 {
4     int i=l,j=m+1,k=l,temp[100];
5     while(i<=m&&j<=h)
6     {
7         if(a[i]<=a[j])
8             temp[k++]=a[i++];
9         else
10            temp[k++]=a[j++];
11     }
12     while(i<=m)
13         temp[k++]=a[i++];
14     while(j<=h)
15         temp[k++]=a[j++];
16     for(i=l;i<=h;i++)
17         a[i]=temp[i];
18 }
19 void mergeSort(int a[],int l,int h)
20 {
21     if(l<h)
22     {
23         int m=(l+h)/2;
24         mergeSort(a,l,m);
25         mergeSort(a,m+1,h);
26         merge(a,l,m,h);

```

Output

```

Enter number of data records: 5
Enter dataset values:
5 7 1 4 2
Merge Sort:
1 2 4 5 7
Quick Sort:
1 2 4 5 7
=== Code Execution Successful ===

```

REPORT ANALYSIS:

1. Problem Statement

A government analytics platform processes large datasets containing public information such as population statistics, demographic records, and survey data. Efficient sorting algorithms are required to organize data for analysis and reporting.

This case study compares **Merge Sort** and **Quick Sort** using the Divide and Conquer approach.

2. Objective

- To implement Merge Sort and Quick Sort.
- To compare their performance.
- To analyze memory usage, stability, and execution speed.
- To select the most suitable sorting algorithm for large datasets.

3. Problem Understanding

Large government datasets require efficient sorting before generating reports.

Input:

- Number of data records
- Dataset values

Output:

- Sorted data using Merge Sort
- Sorted data using Quick Sort

4. Algorithms Used

Merge Sort

Merge Sort divides the dataset into smaller parts until single elements remain. Then it merges the elements in sorted order.

Steps:

1. Divide the array into two halves.
2. Sort each half recursively.
3. Merge the sorted halves.
4. Produce the final sorted dataset.

Quick Sort

Quick Sort selects a pivot element and divides data around the pivot.

Steps:

1. Select pivot element.

2. Place smaller elements before pivot.
3. Place larger elements after pivot.
4. Apply the same process recursively.

5. Comparison Between Merge Sort and Quick Sort

Feature	Merge Sort	Quick Sort
Technique	Divide and Conquer	Divide and Conquer
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n \log n)$	$O(n^2)$
Memory Usage	More memory required	Less memory required
Stability	Stable	Not stable
Speed	Consistent performance	Faster in practice

6. Performance Analysis

Merge Sort

Advantages:

- Guaranteed $O(n \log n)$ performance.
- Suitable for very large datasets.
- Maintains original order of equal elements.

Disadvantage:

- Requires additional memory.

Quick Sort

Advantages:

- Requires less memory.
- Faster for smaller datasets.
- Efficient for practical applications.

Disadvantage:

- Worst-case performance can become $O(n^2)$.

7. Suitable Solution

For government analytics systems with large-scale data processing, **Merge Sort is recommended** because:

- It provides consistent performance.
- It handles large datasets efficiently.
- It maintains data stability during sorting.

Quick Sort can be selected when memory efficiency and faster average execution are more important.

8. Conclusion

Both algorithms use the Divide and Conquer method. Binary Search provides efficient searching in railway reservation systems, while Merge Sort and Quick Sort help organize large government datasets. The selection of an algorithm depends on requirements such as speed, memory usage, and data size.